NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

JOÃO EDUARDO GOMES PIRES GARÇÃO

BSc in Computer Science and Engineering

AN INTEGRATED FRAMEWORK FOR THE DEVELOPMENT OF CRDTS

SOME THOUGHTS ON THE LIFE, THE UNIVERSE, AND EVERYTHING ELSE

Dissertation Plan
MASTER IN COMPUTER SCIENCE AND ENGINEERING

NOVA University Lisbon

Draft: *February 7, 2026*

AN INTEGRATED FRAMEWORK FOR THE DEVELOPMENT OF CRDTS

SOME THOUGHTS ON THE LIFE, THE UNIVERSE, AND EVERYTHING ELSE

JOÃO EDUARDO GOMES PIRES GARÇÃO

BSc in Computer Science and Engineering

Supervisor: Mário José Parreira Pereira
Associate Professor, NOVA University Lisbon

Programme Coordinator: Carla Maria Gonçalves Ferreira
Full Professor, NOVA University Lisbon

Dissertation Plan
MASTER IN COMPUTER SCIENCE AND ENGINEERING

NOVA University Lisbon
Draft: *February 7, 2026*

Abstract

Conflict-Free Replicated Data Types (CRDTs) are a key abstraction for modern distributed systems, allowing data to be updated concurrently across multiple replicas while ensuring Strong Eventual Consistency without centralized coordination. Despite these guarantees, the correct design of CRDTs remains challenging and is prone to errors that may compromise data integrity. Automated verification tools based on SMT solvers, support rapid prototyping and validation of convergence properties, but they face significant limitations when reasoning about recursive definitions, inductive properties, and complex global state invariants, often resulting in inconclusive verification outcomes.

This dissertation addresses these challenges by proposing an integrated framework for the modular and formal development of CRDTs that combines several deductive-based backend tools. The framework leverages the automation provided by VeriFx for checking algebraic convergence properties, while relying on the expressive power of the Why3 platform to ensure correctness guarantees through explicit contracts and invariants. A formal translation from VeriFx specifications to WhyML is introduced, enabling the use of deductive verification to enforce correct-by-construction semantics where automation alone is insufficient. Preliminary work, from translating and verifying fundamental CRDTs such as the Positive-Negative Counter in both VeriFx and Why3, indicate that specifications can be interchanged between tools without loss of correctness. The translation reveals the invariants and structural properties that need to be made explicit in a deductive setting, motivating the integration of multiple verification techniques.

Keywords: CRDTs · Formal Verification · Why3 · VeriFx

Resumo

Os Conflict-Free Replicated Data Types (CRDTs) representam uma abstração fundamental para sistemas distribuídos modernos, permitindo a atualização concorrente de dados em múltiplas réplicas e assegurando uma Consistência Eventual Forte sem coordenação centralizada. Apesar destas garantias, a construção correta de CRDTs continua a ser um desafio propenso a erros devido a poder comprometer a integridade dos dados. Embora as ferramentas de verificação automática baseadas em solvers SMT, permitam a criação rápida de protótipos e a validação de propriedades de convergência, estas enfrentam limitações significativas ao lidar com definições recursivas, propriedades indutivas e invariantes de estado global complexos, resultando frequentemente na impossibilidade de concluir a verificação.

Esta dissertação responde a estes desafios propondo uma framework integrada para o desenvolvimento modular e formal de CRDTs, que combina diversas ferramentas de backend baseadas em dedução. A framework tira partido da automação fornecida pelo VeriFx para verificar propriedades algébricas de convergência, enquanto recorre ao poder expressivo da plataforma Why3 para assegurar correções através de contratos e invariantes explícitos. É apresentada uma tradução formal das especificações VeriFx para WhyML, permitindo o uso de verificação dedutiva para impor uma semântica correta por construção onde a automação isolada é insuficiente. O trabalho preliminar, que incluiu a tradução e verificação de CRDTs fundamentais como o Positive-Negative Counter tanto no VeriFx como no Why3, indica que as especificações podem ser partilhadas entre ferramentas sem perda de correção. A tradução revela os invariantes e propriedades estruturais que necessitam de ser explicitados num ambiente dedutivo, motivando a integração de múltiplas técnicas de verificação.

Palavras-chave: CRDTs · Verificação Formal · Why3 · VeriFx

Contents

List of Figures

List of Tables

Listings

1 Introduction

1.1 Motivation

Modern computing is inherently dependent on large-scale distributed systems, where high availability and low latency are essential requirements. To meet these demands in the presence of network failures and temporary disconnections, the CAP theorem [8] implies that strong consistency must often be relaxed in favor of availability. In this context, Conflict-Free Replicated Data Types have emerged as a fundamental solution, providing Strong Eventual Consistency by construction [18]. CRDTs allow distributed replicas to be updated independently and concurrently, while guaranteeing that all replicas eventually converge to the same state without requiring synchronous coordination.

Despite these guarantees, implementing CRDTs correctly is challenging and prone to errors that can compromise data integrity. Consequently, formal verification techniques play an important role in increasing confidence in their correctness. Existing approaches can be divided into automated verification and deductive verification. Automated tools based on SMT solvers, such as VeriFx [14], support the systematic validation of convergence properties and the automatic discovery of counterexamples. Deductive verification platforms, such as Why3 [7], offer stronger guarantees by enabling proofs of functional correctness through contracts and invariants.

Experience shows that neither approach is sufficient when used in isolation. Automated verification, while efficient, struggles to handle complex or recursive structures. Deductive verification, although more expressive, often requires manual effort. This highlights the need for a methodology that combines the strengths of both approaches to support the verification of modern CRDTs.

1.2 Problem Definition

Despite advances in automated verification, tools such as VeriFx face clear limitations when applied to CRDTs with more complex behavior. The main problem addressed in this dissertation is that purely automated techniques are often unable to guarantee

correctness in scenarios that require inductive reasoning or the preservation of global state invariants [17].

Previous work has shown that SMT-based solvers exhibit well-known limitations when reasoning about recursive definitions [7]. First, SMT-based solvers have difficulty reasoning about recursive definitions. Properties that depend on iterating over data structures of unbounded size are hard to prove without explicit induction. This issue arises, for example, in vector-based counters, where computing the observable value requires recursion over a structure whose size depends on the number of replicas, often leading to timeouts or inconclusive results.

Another limitation of automated verification is that it does not guarantee that the implemented conflict-resolution policy aligns with the intended semantics. Convergence alone cannot distinguish between different behaviors, such as add-wins or remove-wins. In addition, verification results may be unstable, as small, non-functional code changes can lead to unexpected verification failures. These limitations show that automation alone is insufficient for verifying modern CRDTs, motivating the use of deductive techniques that can reason about induction, enforce invariants, and provide stronger semantic guarantees.

1.3 Goals and Expected Contributions

This work aims to explore how automated and deductive verification techniques can be combined to support the formal development of Conflict-Free Replicated Data Types. By leveraging the strengths of both approaches, it seeks to address the limitations observed when each technique is applied in isolation.

The expected contributions of this work are threefold. First, it delivers a formally verified library of CRDTs implemented in VeriFx and Why3, covering counters, sets, and register-based data types, and including both state-based and operation-based models. This library provides verified implementations of representative CRDT designs under different verification paradigms. Second, the work introduces a translation methodology from VeriFx specifications to WhyML [19], defining clear rules for converting convergence-oriented specifications into formal contracts and invariants suitable for deductive verification. Finally, an experimental evaluation demonstrates that the integrated approach addresses key limitations of purely automated verification, particularly in the presence of recursive definitions and global state invariants, and enables correct-by-construction reasoning in scenarios where SMT-based methods alone are insufficient.

Within this context, the specific goals of the dissertation are as follows:

- To integrate distinct verification methodologies by investigating and formalizing the complementarity between VeriFx and Why3.
- To guarantee the correct-by-construction property by developing mechanisms that go beyond checking eventual convergence and instead enforce strong state invariants ensuring semantic validity at every execution state.

- To overcome expressiveness limitations in complex algorithms by addressing the verification of data structures that rely on recursive definitions or require inductive reasoning.

1.4 Report Structure

The remainder of this document is organized as follows:

- Chapter 2 presents the state of the art on consistency models, CRDT theory, and existing formal verification tools.
- Chapter 3 details the theoretical foundations required for this work, including the formalization of CRDTs, the logic underlying SMT solvers, and the principles of deductive verification with Why3.
- Chapter 4 describes the preliminary work carried out so far, including the implementation and verification of several CRDTs in VeriFx, the identification of practical limitations, and the successful translation and verification of selected case studies in Why3.
- Chapter 5 outlines the work plan for completing the dissertation, including the task schedule for consolidating the proposed framework.

2

Related Work

In this chapter, the relevant concepts and related work on consistency, replication, conflict resolution in distributed systems, and verification methods are presented. It starts by introducing consistency and replication models, clarifying the trade-offs between strong and weak consistency. The chapter then discusses how divergent replicas are handled, covering both ad-hoc conflict resolution strategies and principled approaches based on Conflict-free Replicated Data Types. Then, the synchronization models for CRDTs and the challenges that arise when composing and nesting replicated data structures are discussed. Finally, it addresses the role of formal verification in ensuring correctness, reviewing different verification approaches and the tools commonly used in practice, along with their capabilities and limitations.

2.1 Consistency models and Replication

Consistency models are commonly used to characterize the behavior of a system when multiple write and read operations are executed concurrently, defining the conditions under which different results can be observed. By establishing these rules, the model determines the level of consistency that applications can rely on. However, stronger consistency guarantees often come at the cost of reduced availability provided by the system, as established by the CAP theorem [8].

The CAP theorem states that a distributed system can not simultaneously provide strong consistency and high availability in the presence of network partitions. Since network partitions are unavoidable in practice, systems must adopt different trade-offs depending on the perpouse of the system.

Under strong consistency assumptions, it is considered that once an operation completes, all replicas reflect a uniform state, ensuring that clients observe the most recent write regardless of the node accessed. Alternatively, approaches that prioritize availability relax these guarantees to maintain availability during failures.

2.1.1 Weak Consistency

Systems that adopt a weak consistency model prioritize high availability and low latency when responding to client requests. In this model, client operations are executed on a small subset of replicas, which execute the operation locally and reply to the client before the update is propagated to the remaining replicas. This propagation occurs asynchronously, avoiding the need for coordination with a majority of replicas, resulting in lower latency and higher availability. However, the relaxed coordination requirements among replicas lead to divergencies, where different replicas can return different values for the same read operation. While these systems may eventually converge, the temporary divergence between replicas increases the complexity of reasoning about system behavior, making application development and debugging more challenging [15].

Eventual Consistency is a weak consistency model that prioritizes availability over consistency guarantees. Under this model, replicas are allowed to diverge temporarily, with the only guarantee being that, in the absence of further updates, all replicas will eventually converge to a common state. During this convergence period, clients may observe out-of-date or inconsistent values across replicas.

To ensure convergence, eventual consistent systems rely on additional mechanisms for resolving conflicting updates.

Causal Consistency is one of the strongest weak consistency models that can be achieved while maintaining availability. This model requires the system to maintain causal dependencies between operations and guarantees that all clients observe causally related operations in an order that respects these dependencies.

Two operations are causally related when the execution of one can influence the execution of the other. For example, in a social networking application, if a user creates a post and subsequently deletes it, the delete operation is causally dependent on the create operation and must be observed after it. Operations that are not causally related are considered concurrent and may be observed in different orders by different replicas. With this enforced consistent ordering of causally related operations while allowing flexibility in the ordering of concurrent ones, causal consistency provides higher consistency guarantees than eventual consistency without sacrificing availability [4].

Causal+ Consistency extends causal consistency by ensuring that replicas converge to the same state even in the presence of concurrent updates. Although causal consistency preserves the ordering of causally related operations, it allows concurrent operations to be applied in different orders, potentially leading to divergent replica states. Causal+ consistency overcomes this issue by enforcing a convergent conflict resolution strategy, in which concurrent operations are merged using associative

and commutative functions, guaranteeing eventual convergence while preserving causal dependencies [18].

2.1.2 Strong Consistency

Systems that adopt a strong consistency model prioritize providing a single, well-defined view of the system state to all clients. In these systems, client operations are executed in a coordinated manner across replicas, ensuring that read operations always observe the effects of previously completed writes according to a well-specified ordering.

To enforce this guarantee, strong consistency models require tighter coordination among replicas, often involving synchronous communication and agreement protocols before operations are considered complete. While this coordination simplifies reasoning about system behavior and enables applications to rely on intuitive semantics, it typically comes at the cost of increased latency and reduced availability when compared to weakly consistent systems.

Linearizability is a strong consistency propertie that enforces a strict ordering of operations in replicated systems. It requires that each operation on a shared object appears to take effect at a single, indivisible point in time between its invocation and completion. Consequently, the outcome of all operations must be consistent with a global order that reflects the real-time order in which operations are issued by clients.

This propertie guarantees that once a write operation completes, its effects are immediately visible to all subsequent operations, independently of the replica they access.

Serializability is a consistency propertie that considers the execution of transactions, each composed of one or more operations that may access multiple objects. It guarantees that the execution of a set of concurrent transactions is equivalent to a serial execution, where transactions are totally ordered.

With this propertie, all replicas observe transactions in the same order. However, this order is not required to respect real-time constraints. Therefore, the serialization order may differ from the temporal order in which transactions are issued according to a global wall clock.

Sequential Consistency is a consistency propertie in which all clients observe operations as if they were executed in a single global order. This order is shared by all replicas, ensuring that every client perceives the same sequence of operations, even if this sequence does not correspond to the real-time order in which operations are issued.

While the global execution order does not need to respect real-time constraints, it must preserve the program order of each individual client or process. That is, if a

process issues an operation op1 before op2, all replicas must observe op1 preceding op2 in the global sequence.

Sequential consistency operates with individual operations. For this reason, systems that guarantee serializability also satisfy sequential consistency, whereas the opposite is not true. By enforcing a common operation order across replicas, sequential consistency prevents clients from observing divergent states when accessing different replicas.

2.1.3 Strong Eventual Consistency

Strong Eventual Consistency characterizes a class of replication models in which replicas are allowed to execute updates independently and propagate them asynchronously, without relying on coordination or global ordering. The central guarantee of this model is that any replicas that have applied the same set of updates will deterministically converge to an identical state, regardless of the order in which those updates are delivered or executed [18].

This model is typically defined by three properties. `Eventual Delivery`, ensures that all updates are eventually propagated to all correct replicas. `Strong Convergence` guarantees that replicas that have applied the same set of updates reach an identical state. `Causal Consistency` preserves the ordering of causally related updates. Together, these properties prevent permanent divergence while avoiding the coordination overhead required by strongly consistent models.

Strong Eventual Consistency directly addresses the limitations of weakly consistent systems by eliminating non-deterministic conflict resolution, while retaining high availability and low latency under network partitions. In practice, strong eventual consistency is achieved through Conflict-free Replicated Data Types, whose design ensures that concurrent updates can be merged deterministically, enabling convergence without coordination [15].

2.2 Conflict Resolution Strategies

In practice, many replicated systems adopt weak consistency models in order to achieve high availability and low latency, allowing replicas to diverge temporarily as updates are executed independently and propagated asynchronously. While this design choice improves system responsiveness, it introduces the possibility of conflicting updates, making conflict resolution a fundamental requirement to obtain a consistent state.

Conflict resolution strategies define how concurrent or divergent updates are reconciled once replicas exchange their states or operations. Historically, these strategies have been broadly classified into ad-hoc approaches, which resolve conflicts using application-specific or heuristic rules, and principled approaches based in formal properties, such as

those employed by Conflict-free Replicated Data Types. These approaches have a direct impact on system semantics, predictability, and scalability.

Consequently, conflict resolution is closely linked to the underlying replication and consistency model, influencing not only correctness guarantees but also the degree of coordination required and the overall system behavior under concurrency and failures.

2.2.1 Ad-hoc Strategies

Ad-hoc conflict resolution strategies resolve conflicts using simple, often application-specific rules, without providing formal guarantees of deterministic convergence in the presence of concurrent updates, as they may lead to undesirable behaviors.

Last-Writer-Wins is one of the most common conflict resolution policies. Under this strategy, concurrent updates are resolved by selecting the update with the most recent timestamp or highest value according to a predefined global ordering.

Although Last-Writer-Wins is simple to implement and introduces minimal coordination overhead, it is prone to lost updates, as concurrent modifications may overwrite each other without being merged. For example, in a replicated shopping cart, concurrent additions of different items by multiple users may result in only one of the updates being reflected in the final state, leading to unintended data loss [18].

Programmatic conflict resolution delegates the responsibility of resolving conflicts to the application logic. This approach was popularized by systems such as Amazon Dynamo [6], where replicas may return multiple conflicting versions of the same object to the client, which is then responsible for reconciling them and writing back a resolved version. While this strategy provides flexibility and allows applications to define domain-specific resolution semantics, it places a significant load on developers. Designing correct and efficient merge logic is often complex, particularly in the presence of high concurrency, and errors in conflict resolution may lead to inconsistent or unintended application states [10].

2.2.2 Conflict-free Replicated Data Types

Conflict-free Replicated Data Types emerge as a rigorous alternative for addressing replica divergence without the drawbacks associated with ad-hoc conflict resolution strategies. Rather than resolving conflicts through heuristic or application-specific rules, CRDTs embed conflict resolution directly into the data type design, ensuring predictable and well-defined behavior under concurrent updates.

A CRDT is an abstract data type designed to be replicated across multiple nodes, allowing replicas to perform local updates without requiring coordination. Convergence is ensured through mathematical principles, such as the commutativity and associativity of operations or the use of state-based merge functions defined over join-semilattices [18].

Therefore, any two replicas that have applied the same set of updates are guaranteed to reach an identical state deterministically, regardless of message ordering or network delays.

CRDTs include a variety of classes designed for different application needs, including counters, registers, sets, and graphs. Some of these data types exhibit recursive structures, where operations may depend on nested or hierarchical elements, enabling the design of more complex CRDTs that maintain convergence guarantees even in recursive scenarios [2].

In contrast to ad-hoc strategies such as Last-Writer-Wins, CRDTs ensure that all concurrent updates are reflected in the final state, avoiding unintended data loss. Since CRDTs do not rely on consensus protocols in the critical path of update execution, they exhibit high scalability and fault tolerance. Updates can be applied with low latency and propagated asynchronously, enabling systems based on CRDTs to remain available and responsive even in the presence of network partitions.

2.2.3 Hybrid and Tunable Policies

While standard CRDTs provide strong convergence guarantees, they typically enforce a fixed conflict resolution policy, such as add-wins or remove-wins semantics, which may not be suitable for all application scenarios. In practice, applications often require policies that depend on the context, for example, temporary disconnections may favor add-wins semantics to preserve availability, while explicit actions such as removals or bans may require remove-wins semantics to ensure they take effect. To address this limitation, hybrid and tunable approaches have been proposed [16], allowing conflict resolution semantics to be adapted for individual updates. Tunable CRDTs enable developers to select the desired resolution policy for each update, rather than committing to a single global policy.

2.3 Synchronization Models of CRDTs

The synchronization model defines the assumptions and guarantees required by a system to ensure the correct behavior of CRDTs in a replicated environment. In particular, it specifies how updates are propagated among replicas and under which conditions convergence is achieved, ensuring that all replicas that receive the same set of updates eventually reach an identical state. To this end, CRDTs rely on two fundamental synchronization models, state-based replication, in which replicas periodically exchange their full state, and operation-based replication, where individual updates are disseminated and applied at remote replicas [18].

2.3.1 Convergent Replicated Data Types

Convergent Replicated Data Types adopt a state-based synchronization model, in which replicas periodically propagate their entire local state to other replicas. Upon receiving a remote state, a replica integrates it with its own state through a deterministic merge function, ensuring that both replicas advance towards a common state. This approach decouples local updates from coordination, allowing replicas to apply operations independently and reconcile divergences through state exchange.

Convergence in CvRDTs is achieved by constraining how replica states evolve and how divergent states are reconciled. The set of possible states is partially ordered, allowing replicas to compare states in terms of the information they contain. Local updates are required to be monotonic, meaning that applying an update to a state always produces a new state that is greater than or equal to the previous one. When replicas exchange state, reconciliation is performed by a deterministic merge function that computes the least upper bound of the two states, preserving all information observed by either replica. This merge operation is necessarily commutative, associative, and idempotent, ensuring deterministic convergence despite message reordering, duplication, or frequency of state exchanges.

From a systems perspective, CvRDTs impose minimal requirements on the underlying network. They tolerate message loss, duplication, and reordering, assuming only that communication is eventually reliable and that replicas can re-establish connectivity after partitions. This robustness makes CvRDTs particularly well-suited for highly unreliable or environments subject to network partitions. However, this model may incur significant communication overhead, as entire states must be transmitted during synchronization, which can become inefficient for frequently updated objects.

2.3.2 Commutative Replicated Data Types

Commutative Replicated Data Types adopt an operation-based synchronization model, in which replicas propagate individual update operations rather than exchanging their full state. Each update is executed locally and disseminated to other replicas, allowing changes to be applied incrementally and avoiding the transmission of large state payloads.

When an update is issued, it is first processed at the origin replica, where any necessary context is used to produce an operation that can be safely applied at other replicas. This operation is then disseminated to the remaining replicas, where it is applied to the local state of each one. Therefore, replicas can generate updates independently, while ensuring that all replicas eventually apply the same operation and reach an identical state.

Convergence in CmRDTs relies on the commutativity of update operations. Since replicas may receive and apply operations in different orders, all operations that can be executed concurrently are required to commute, ensuring that the order in which updates are applied does not affect the final state. Once all replicas have received the same set of

operations, they deterministically converge to an identical state, without requiring global coordination or synchronization.

Compared to state-based CRDTs, CmRDTs place stronger requirements on the underlying communication channel. Operations must be reliably delivered to all replicas, typically with causal ordering guarantees, to ensure that concurrent updates commute as expected and replicas do not diverge. While this increases the complexity of the communication layer, it significantly reduces synchronization overhead and makes CmRDTs more efficient for objects with large state or high update rates.

2.3.3 Composition and Nesting

While the properties of basic CRDTs are well understood, practical systems frequently require combining these primitives into more complex, hierarchical structures, such as maps containing nested objects, JSON documents, or directory trees. In these settings, simply nesting CRDTs is not sufficient, as the convergence guarantees of individual data types do not automatically extend to the composed structure. A challenge arises when concurrent operations affect different levels of the hierarchy, for instance, when an update modifies a nested element while a concurrent operation removes or replaces its parent.

These challenges have been studied in the context of replicated JSON documents [9], which are often regarded as a reference model for conflict resolution in deeply nested data. CRDT based approaches to JSON highlight that simple resolution policies, such as Last-Writer-Wins, are insufficient to preserve user intent in hierarchical structures, as they may discard meaningful concurrent updates. Consequently, specialized mechanisms are required to reconcile edits across multiple levels of the hierarchy while maintaining deterministic convergence.

To address these issues, several mechanisms for combining CRDTs have been proposed. One commonly adopted approach is recursive reset semantics, where removing an element from a collection implicitly resets or discards all nested state. More recent approaches generalize this idea by explicitly capturing the causal relationships between parent and child objects. In particular, nested operation-based CRDT models [2] introduce mechanisms that allow removals or resets to be applied selectively and recursively, based on causal information, ensuring that the semantics of the enclosing structure are preserved, providing deterministic convergence for complex, hierarchical CRDTs without introducing excessive metadata overhead.

2.4 Formal Verification

Formal verification of replicated data types is essential because it is difficult to reason about the many possible execution orders of concurrent operations in distributed systems. Subtle combinations of concurrent updates, message reordering, or partial failures can lead to unexpected behaviors that are hard to detect through testing alone. Therefore,

informal reasoning or ad-hoc validation techniques are often insufficient to guarantee correctness [4].

Research in verification of replicated data types has evolved from manual, pen-and-paper proofs toward automated and deductive verification techniques. Early approaches relied heavily on human reasoning to establish convergence and correctness properties, while more recent work has focused on developing formal frameworks and tool-supported methods that provide stronger guarantees and better scalability [14].

2.4.1 Manual Verification

Early efforts to validate CRDTs relied primarily on manual, pen-and-paper proofs. These proofs aimed to establish key properties such as convergence and correctness by reasoning about the abstract behavior of the data type under concurrency. From a theoretical perspective, this approach offers a high level of rigor, as it allows precise arguments to be constructed over well-defined formal models.

However, manual verification presents significant limitations. Constructing such proofs is time consuming and requires substantial expertise in formal methods, making the approach difficult to scale beyond a small number of data types. Moreover, these proofs often reason about an abstract specification of the algorithm, which may differ from its concrete implementation. Consequently, even when the specification is formally correct, errors may still occur in the implementation.

2.4.2 Automated Verification

To make formal verification more accessible to non-specialists, several automated verification approaches have been proposed, often based on high-level specification languages. One such example is VeriFx, a framework that allows replicated data types to be implemented and automatically verified for Strong Eventual Consistency using SMT solvers such as Z3. By abstracting away low-level proof details, these tools significantly reduce the complexity of verifying replicated data structures and have been successfully applied to a wide range of basic CRDTs.

Despite these advances, the literature identifies important limitations in existing automated verification tools. In particular, handling recursive algorithms remains challenging, as demonstrated by the inability of VeriFx to verify data types such as the Replicated Growable Array, whose correctness depends on recursive insertion logic. Furthermore, the verification of more complex objects, especially those relying on version vectors, can be unstable in practice. For instance, small and semantically irrelevant changes to a specification, such as renaming variables, may cause the solver to fail or abort the proof process, highlighting limitations in robustness and predictability.

2.4.3 Deductive Verification

To overcome the limitations of purely automated verification tools, deductive verification frameworks provide an alternative approach. One such framework is Why3 [7], which is based on the WhyML [19] specification language and first-order logic. Why3 generates proof obligations that can be discharged by a variety of external theorem provers, allowing verification tasks to be expressed in a structured and modular way.

2.4.4 Invariants and Security

While the formal verification of CRDTs traditionally focuses on structural convergence, such as ensuring that merge operations form a join-semilattice, the correctness of a distributed application often depends on application-level invariants, including access control policies and other security-related constraints that extend beyond the data type itself.

This challenge has been studied in the context of distributed file systems operating under local-first assumptions, where security requirements are commonly expressed in terms of confidentiality, integrity, and availability. Previous work in this area has shown that conventional access control models, such as those inspired by POSIX, give rise to conflicts when replicas operate independently and synchronize asynchronously. These studies establish an impossibility result analogous to the CAP theorem, in this systems that prioritize high availability and tolerate network partitions, it is not possible to simultaneously guarantee confidentiality, integrity, and full availability during conflict resolution, meaning that one of these properties must be relaxed when reconciling concurrent updates [20].

2.5 Verification Tools

Verification tools for Replicated Data Types differ in how they balance automation, expressiveness, and proof stability. Initial approaches were based on highly expressive interactive proof assistants, such as Isabelle/HOL [13], which offer strong guarantees but require substantial expertise and manual effort. More recent tools adopt SMT-based automation [14] to simplify the verification process at the cost of reduced expressiveness and, in some cases, stability.

2.5.1 VeriFx

VeriFx [14] represents a significant step toward making the formal verification of CRDTs accessible. It provides a high-level language, inspired by Scala, in which developers can implement replicated data types using functional abstractions, while automatically verifying properties such as Strong Eventual Consistency through SMT solving with *Z3* [12].

A key strength of VeriFx lies in its ability to connect specification, verification, and deployment. Designs that are proven correct at the specification level can be translated into executable implementations in languages such as Scala and JavaScript, reducing the gap between formally verified models and practical systems.

Despite these advantages, VeriFx presents several fundamental limitations that restrict its applicability to more complex designs. The tool struggles with recursive algorithms, failing, for instance, to verify the Replicated Growable Array due to the recursive nature of its insertion logic, which exceeds the solver's ability to reason about unbounded execution paths. In addition, VeriFx exhibits a pronounced sensitivity to syntactic changes, where minor modifications, such as renaming a variable, can cause previously successful proofs to fail or viceversa. Finally, VeriFx does not fully capture the semantic intent of certain conflict resolution policies, as it may prove convergence for a data type even when the verified semantics, such as remove-wins instead of add-wins, do not align with the programmer's intended behavior.

In addition to these tool-specific limitations, SMT-based verification approaches face significant challenges when reasoning about nested and recursive CRDTs. In such cases, solvers may fail to converge within finite time, forcing developers to resort to manual reasoning techniques, such as structural induction, to establish correctness properties.

2.5.2 Why3

Why3 [7] is a platform for deductive program verification based on a first-order specification and programming language called WhyML [19]. Rather than relying on a single automated solver, Why3 serves as an intermediate verification layer that generates proof obligations and dispatches them to multiple external provers, such as Alt-Ergo [5], Z3 [12], and CVC [1].

Compared to purely SMT-based approaches, Why3 offers greater expressiveness and stability when reasoning about complex data structures. Its deductive verification model is particularly well-suited for nested or recursive definitions, which are difficult to handle using direct SMT encodings. In addition, Why3 enforces a static discipline over mutable state and aliasing, avoiding the need for intricate memory models and resulting in proof obligations that are easier to reason about and maintain.

As a consequence, Why3 provides a more flexible and controlled verification environment than fully automated tools such as VeriFx. While this approach requires more user guidance and interaction, it enables the verification of a broader class of replicated data types and offers a solid basis for addressing limitations related to recursion, modularity, and proof stability.

3

Background

This chapter establishes the theoretical foundations essential for the work developed in this dissertation. It starts by presenting the formalization of Conflict-free Replicated Data Types, detailing the synchronization models based on state and operations, as well as the role of causality in ensuring convergence. It then explores VeriFx, describing its approach to CRDT modeling and its automatic verification mechanism based on SMT solvers. Finally, it introduces the Why3 platform and the WhyML language [19], highlighting the support for deductive verification through explicit invariants and ghost code, which are fundamental for overcoming the limitations of purely automated approaches.

3.1 Formalization of CRDTs

The correctness of CRDTs is achieved through strong mathematical properties that ensure convergence across replicas, rather than through heavy coordination mechanisms. Regardless of message reordering, duplication, or temporary network failures, replicas that apply the same set of updates are guaranteed to reach an identical state. This section presents a formal view of CRDTs by characterizing the two fundamental synchronization models, the state-based model CvRDTs, and the operation-based model CmRDTs, as well as the notion of causality that supports their correctness guarantees.

3.1.1 State Based Convergence

State-based CRDTs ensure convergence by defining their state space as a join-semilattice [18]. Formally, a CvRDT is defined by a triple $(S, \leq, \sqcup)$, where S denotes the set of possible states, $\leq$ is a partial order over S, and $\sqcup : S \times S \rightarrow S$ is a merge operator that computes the least upper bound of two states.

To guarantee deterministic convergence, the merge operation must satisfy three algebraic properties for all states $s_1, s_2, s_3 \in S$, to ensure that merging replicas yields the same result regardless of message ordering, duplication, or repetition. These properties are: associativity $(s_1 \sqcup s_2) \sqcup s_3 = s_1 \sqcup (s_2 \sqcup s_3)$, commutativity $s_1 \sqcup s_2 = s_2 \sqcup s_1$, and idempotence, $s_1 \sqcup s_1 = s_1$.

In addition, all local update operations must be monotonic with respect to the partial order $\leq$. That is, if a state s is updated by an operation u, producing a new state s', it is required that $s \leq s'$. This monotonicity property ensures that replicas always progress within the lattice, preventing regressions that could otherwise lead to divergence in asynchronous systems. The function $compare(x, y)$, used in pratical implementations, corresponds to the partial order relation, which returns true if $x \leq y$. This relation is then used to define state equivalence, where two states are considered equivalent if and only if $x \leq y$ and $y \leq x$.

3.1.2 Operation Based Convergence

Operation-based CRDTs achieve convergence by disseminating update operations rather than exchanging full states. In this model, an update is first processed at the source replica, where its preconditions are checked and the corresponding operation is generated. This operation is then propagated through the system and applied to the local state of all replicas, ensuring that each replica eventually executes the same set of updates.

A sufficient condition for convergence in a CmRDT is that all concurrent operations commute [18]. Given two operations f and g that are concurrent, applying them in any order must yield the same resulting state $S \cdot f \cdot g \equiv S \cdot g \cdot f$. This commutativity property allows the replication system to relax the delivery order of concurrent messages, enforcing causal delivery, and ensuring that operations are applied in an order consistent with the happens-before relation, while allowing concurrent updates to be delivered in any order.

3.1.3 Pure Operation-Based CRDTs

Traditional operation-based CRDTs enable the generation of an update to inspect the current state of a replica to produce the message that will be disseminated. Pure operation-based CRDTs [2] adopt a more restrictive approach, in which the propagated message contains only the original operation and its arguments, without including any additional metadata derived from the replica state.

In this model, the state of a replica is represented as a partially ordered log that records the applied operations, together with their causal ordering. Rather than computing the state directly, replicas derive the current state from this log.

To control the size of the log, this model relies on two concepts, the first being causal redundancy, where a newly delivered operation makes the effects of earlier operations no longer relevant. For example, a remove operation can remove the effect of a previous add of the same element, allowing the earlier operation to be safely ignored. Identifying such cases enables the system to discard redundant operations from the log without altering the resulting state.

The second concept is causal stability. When it is known that all replicas have observed an operation and that no concurrent operations remain, the causal information associated

with that operation is no longer needed. This allows replicas to discard causal metadata while still guaranteeing deterministic convergence.

3.1.4 Causality and Logical Clocks

In distributed systems without centralized coordination, the notion of time is defined through the happens-before relation, originally introduced by Lamport [11]. This relation plays a central role in CRDTs, as it allows the system to distinguish between sequential and concurrent events, which is essential for data types whose semantics depend on concurrency, such as add-wins sets or multi-value registers.

Causality establishes that an event a occurs before an event b when a and b take place at the same replica and a precedes b, for example, when a corresponds to the sending of a message and b to the reception of that message, or when the ordering follows transitively from other events. If neither $a \rightarrow b$ nor $b \rightarrow a$ holds, the two events are considered concurrent.

To represent causality, systems commonly rely on version vectors. A version vector associates each replica identifier with an integer counter that records how many updates originated at that replica and are reflected in the current state. When a replica performs a local update, it increments its own counter, and version vectors are merged by taking, for each replica, the maximum of the corresponding counters. This allows causal histories of states or operations to be compared, a version vector is considered causally ahead of another if, for every replica, its counter is greater than or equal to the corresponding counter in the other vector. When neither vector is causally ahead of the other, the associated states are concurrent, indicating a conflict that must be resolved by the CRDT, for example by merging values or applying a predefined resolution policy.

3.2 Modeling and Verification in VeriFx

VeriFx [14] is a high-level programming language designed specifically for the implementation and formal verification of Replicated Data Types. It follows a functional and object-oriented style, restricting programmers to immutable collections and a set of logical constructs, allowing VeriFx programs to be translated into SMT formulas and automatically verified using the Z3 solver [12].

3.2.1 RDT Modeling

In VeriFx, Replicated Data Types are modeled by extending traits provided by the standard library. These traits act as partially implemented interfaces that define the structural and semantic requirements that a data type must satisfy in order to be verified. By constraining implementations to follow these abstractions, VeriFx ensures that the resulting models are suitable for automatic verification of convergence and correctness.

State-based CRDTs are modeled by extending the `CvRDT[T]` trait. This interface requires the implementation of a `merge(that: T): T` method, which defines how two replica states are combined by computing their least upper bound, and a `compare(that: T): Boolean` method, which encodes the partial order over states and is used to reason about equivalence and monotonicity.

Operation-based CRDTs are modeled by extending the `CmRDT[Op, Msg, T]` trait. In this case, updates are represented as operations that are transformed into messages for dissemination and later applied at all replicas. The `prepare(op: Op): Msg` method generates the message corresponding to an update without modifying the local state, while the `effect(msg: Msg): T` method applies a received message to produce the updated state, allowing VeriFx to reason explicitly about the propagation and application of operations when verifying convergence.

3.2.2 Automated Verification

The distinctive feature of VeriFx lies in its support for automated verification directly integrated into the language through the proof construct. During compilation, the VeriFx program and associated proof definitions are translated into logical formulas understood by the Z3 SMT solver. The solver then attempts to determine whether these formulas can be violated by any admissible execution, searching for counterexamples in the space of possible states or operation histories. If no counterexample is found, the specified property is considered to hold for all executions admitted by the model.

Proof Construction defines how correctness properties are expressed and checked in VeriFx. A proof is written as a function without parameters whose body consists of a Boolean expression intended to be universally true. Instead of constructing a proof manually, the verification process attempts to falsify this expression by identifying a concrete execution that contradicts the stated property. This approach, based on counterexamples, enables fully automated reasoning while providing strong guarantees when verification succeeds. Proofs are evaluated over all states and executions that satisfy the constraints imposed by the data type definition, the replication model, and the admissibility conditions encoded in the program.

Convergence Verification focuses on establishing Strong Eventual Consistency through reusable proof abstractions provided by the VeriFx library. For state-based CRDTs, the verifier generates proof obligations that check whether the merge function defined by the user induces a join-semilattice over the set of reachable and compatible states, ensuring idempotence, commutativity, and associativity. For operation-based CRDTs, the verification focuses on ensuring that all concurrent operations commute, the verification engine explores executions in which admissible operations are applied in different orders and checks that the resulting states are equivalent. This

allows convergence properties to be validated systematically across all possible execution orderings admitted by the replication model.

3.3 Deduction and Specification in Why3

Why3 [7] is a platform for deductive program verification that provides a rich specification and programming language, known as WhyML. Instead of relying on a single automated solver, Why3 acts as an intermediate verification framework, it translates specified programs into verification conditions, which are then submitted to a range of external provers, including both automated solvers, such as Z3 [12], CVC [1], and Alt-Ergo [5], and interactive proof assistants, such as Coq [3], allowing users to combine automation with manual reasoning when required.

3.3.1 WhyML

WhyML is designed to act simultaneously as a programming language and as a language for logical specification, allowing executable code and formal properties to be expressed within a single framework, and enabling specifications to be written close to the code described, while still supporting rigorous reasoning by automated and interactive provers. Syntactically, WhyML is close to languages such as OCaml, but enforces some restrictions to ensure that the verification conditions it generates remain manageable for provers and understandable to users.

In particular, WhyML is based on first-order logic, although it supports polymorphism and recursive algebraic data types, reasoning is restricted to first-order constructs, which keeps proofs manageable for automated provers. Higher-order functions are therefore excluded from the logic, even though nested function definitions remain available at programming level.

Another defining aspect of WhyML is the absence of a complex memory model. Rather than explicitly modeling the heap, it enforces static control over aliasing. Each mutable reference has a finite and statically known set of names, and recursive data types are restricted from containing mutable components, preventing the formation of data structures with complex sharing patterns that would complicate logical reasoning.

Mutable state is encapsulated within structures containing mutable fields, allowing the type system to track updates and ensure that state changes are explicitly reflected in the corresponding proof obligations.

3.3.2 Ghost Code and Invariants

Verification in Why3 is based on specifications of program behavior, expressed through preconditions using `requires`, postconditions using `ensures`, and invariants. These specifications are used in functions where they describe the requirements of a method or

function, and the expected behavior upon termination for preconditions and postconditions, respectively. In addition, loops and recursive functions must be included with invariants to capture correctness properties that hold throughout execution, and with variants to establish termination. For the verification of more complex algorithms, such as CRDTs, Why3 provides support for ghost code. Ghost code consists of auxiliary computations and data that are introduced to support formal reasoning and are not taken into account when producing executable code. Such code is constrained so that it cannot influence the control flow of the actual program or modify non-ghost state, ensuring that verification elements do not affect program semantics.

3.3.3 Deductive Proofs

In Why3, the verification process is based on the computation of weakest preconditions to generate proof obligations in the form of logical formulas from a program and its specifications. If these obligations are successfully verified, they guarantee that the program satisfies its specification and is free from certain runtime errors, such as division by zero or out-of-bounds array accesses.

A key advantage of Why3 over approaches that rely solely on SMT solving is its ability to reason about complex inductive and recursive definitions through the integration of interactive provers. This increases robustness and expressiveness when verifying nested or recursive data structures, where fully automated solvers often fail due to timeouts or limitations in handling induction.

4

Preliminary Work

This chapter describes the preliminary experimental work carried out to assess the feasibility of the approach proposed in this dissertation. It begins with the specification and automatic verification of a set of fundamental CRDTs, including counters and sets, using VeriFx to validate convergence properties in both state-based and operation-based models. The chapter then examines the translation of these models into the Why3 deductive verification environment, showing how convergence reasoning can be strengthened through formal contracts and explicit state invariants. Finally, it presents a comparative analysis of the two approaches, highlighting how the addition of deductive verification helps overcome the limitations in expressiveness and robustness of purely automated methods.

4.1 CRDT Verification on VeriFx

In this section, a collection of CRDTs implemented and verified in VeriFx [14] is presented, corresponding to Appendices 5, 5, 5, 5, 5. These implementations cover both state-based and operation-based CRDTs, including counters and set data types, and illustrate how VeriFx is used to automatically verify convergence properties.

4.1.1 State-Based Counters

State-based counters are a simple example of state-based CRDTs. Their implementation in VeriFx allows us to illustrate how convergence is specified and automatically verified, as well as how vector-based state and the composition of Abstract Data Types are handled.

Grow-only Counter constitutes the basis for state-based counters. Its implementation, shown in Appendix 5 and provided by Kevin De Porre [14], follows the formal specification by Shapiro et al. [18], modeling the state as a `Vector[Int]` where each position corresponds to a specific replica.

The `increment` method handles state updates. To prevent write conflicts and ensure monotonic growth, it uses the provided replica identifier to locate and update only

the specific entry assigned to that replica. Convergence is achieved through the `merge` method, which computes the least upper bound of two states. By combining the vectors and taking the maximum of corresponding entries, this method ensures the idempotence, commutativity, and associativity properties required of a join-semilattice. Finally, the observable value is computed by the recursive `computeValue` function, which sums all vector entries. VeriFx validates this logic to confirm that the implementation satisfies the consistency requirements of a convergent CRDT.

Positive-Negative Counter builds upon the previous definition to demonstrate the verification of composed CRDTs. Its implementation, shown in Appendix 5, structures the state by composing two GCounter instances: `p` to track increments, and `n` to track decrements.

The `increment` and `decrement` methods operate on these distinct fields. Specifically, a decrement is modeled as an increment to the `n` counter, leveraging the verified logic of the underlying GCounter. Convergence is achieved through the `merge` method, which simply delegates the synchronization logic to the internal components `p` and `n`. Since VeriFx has already validated the GCounter, the correctness of the PNCounter is automatically inferred from the properties of its constituents. Finally, the `value` method computes the observable state by calculating the difference between the positive and negative accumulations.

4.1.2 State-Based Sets

State-based sets constitute a fundamental category of state-based CRDTs. Their implementation in VeriFx confirms that the use of native set primitives facilitates the automatic proof of join-semilattice properties.

Grow-only Set represents an append-only collection. Its implementation, shown in Appendix 5, models the state using a generic immutable set `Set[V]`, ensuring monotonicity by construction, as elements can be added but never removed.

The `add` operation updates the state by returning a new instance that includes the added element, while convergence is ensured by the `merge` operation, which computes the union of the states of two replicas using `this.set.union(that.set)`. VeriFx automatically verifies that this merge function satisfies the required algebraic properties of commutativity, associativity, and idempotence, illustrating that the underlying SMT solver can reason effectively about set specifications without requiring additional annotations.

Two-Phase Set extends the capabilities of the GSet by supporting element removal, under the restriction that once an element is removed, it cannot be added again.

The implementation, shown in Appendix 5, follows the formal specification by Shapiro et al. [18], using the tombstone technique. The state is represented by two

internal sets, `added`, which stores all added elements, and `removed`, which stores deleted elements.

The `lookup` operation determines visibility based on the logic that an element is considered present only if it exists in `added` and is absent from `removed`. Convergence is ensured by the `merge` operation, which unions the corresponding internal sets. VeriFx confirms that this composition preserves convergence properties, validating that the tombstone mechanism effectively resolves concurrency.

4.1.3 Operation-Based Set

Operation-Based OR-Set implements the Observed-Remove Set following the CmRDT model. Its implementation, shown in Appendix 5, adapts the specification from Shapiro et al. [18] to VeriFx, ensuring Add-Wins semantics.

Listing 4.1 outlines the fundamental methods of this CRDT, which rely on unique identifiers to distinguish between multiple additions of the same element. Each addition is associated with a `Tag`, defined as a combination of a replica identifier and a monotonically increasing counter. Consequently, the CRDT state is not represented as a simple set, but as a map `Map[V, Set[Tag[ID]]]`, where an element is considered present only if its associated set of tags is non-empty.

The implementation separates the expression of user intent from the propagation of updates across replicas. When an element is added, the `add` operation generates a unique `Tag` based on the local counter, while the corresponding downstream effect incorporates this tag into the state and updates the counter to reflect the observed causality. For the removal operation, the source replica inspects its local state and collects only the tags currently associated with the target element, ensuring that the generated message refers exclusively to observed additions. When applied downstream, the removal deletes precisely those tags, leaving other concurrent additions unchanged.

VeriFx automatically verifies that these operations commute, confirming Strong Eventual Consistency. The Add-Wins semantics are ensured by the tag management mechanism, when a removal is concurrent with an addition, it only affects the tags known at its origin. The concurrent addition creates a new unique tag that is not removed, allowing the element to remain in the set.

Listing 4.1: The implementation of a Observed-Remove Set in VeriFx

```
1  class Tag[ID](replica: ID, counter: Int)
2
3  class ORSet[V, ID](myID: ID, counter: Int = 0,
4                  elements: Map[V, Set[Tag[ID]]] = new Map[V, Set[Tag[ID]]]) extends CmRDT[
                       ↪ SetOp[V, ID], SetMsg[V, ID], ORSet[V, ID]] {
5    . . .
6    def add(e: V): SetMsg[V, ID] =
7      new AddMsg(e, new Tag(this.myID, this.counter + 1))
8
```

```
9   def addDownstream(e: V, tag: Tag[ID]) = {
10    val newCounter = if (tag.replica == this.myID) tag.counter else this.counter
11    val tags = this.elements.getOrElse(e, new Set[Tag[ID]])
12    val newTags = tags.add(tag)
13    new ORSet(
14      this.myID,
15      newCounter,
16      this.elements.add(e, newTags)
17    )
18  }
19
20  def preRemove(e: V) = this.lookup(e)
21
22  def remove(e: V): SetMsg[V, ID] =
23    new RemoveMsg[V, ID](e, this.elements.getOrElse(e, new Set[Tag[ID]]))
24
25  def removeDownstream(e: V, tags: Set[Tag[ID]]) = {
26    val observedTags = this.elements.getOrElse(e, new Set[Tag[ID]])
27    val newTags = observedTags.diff(tags)
28    val newElements = this.elements.add(e, newTags)
29    new ORSet(
30      this.myID,
31      this.counter,
32      newElements
33    )
34  }
35  . . .
36 }
37 object ORSet extends CmRDTProof2[SetOp, SetMsg, ORSet]
```

4.2 CRDT Verification in Why3

To illustrate the verification of CRDTs in Why3 [7], the Grow-only and Positive-Negative Counters are translated into WhyML [19]. These example demonstrates how components verified in VeriFx can be formalized within a deductive setting that supports explicit invariants and interactive reasoning. The following discussion analyzes the implementation in Listings 5 and 5, comparing it with the reference code in Listing 5.

4.2.1 Abstract CRDT Module

In Why3, the CRDT module is established through a formal contract within an abstract module, contrasting with the traits used in VeriFx. The definition, shown in Listing 4.2, declares abstract types for the internal state `t` and the observable value `payload`. The module specifies the core operations `merge` and `compare`, with the latter used to define state equivalence through mutual comparison in the `equals` predicate. Correctness properties

are defined as axioms, such as `merge_commutative`, which enforce convergence conditions at the specification level by requiring implementations to provide formal proofs.

Listing 4.2: The implementation of CRDT module in Why3

```
module Crdt

  type payload
  type t

  val function create () : t
  val function get_payload (t: t) : payload

  function merge t t : t

  predicate compare t t

  predicate equals (t1 t2: t)
  = compare t1 t2 /\ compare t2 t1

  axiom merge_commutative: forall t1 t2: t.
    equals (merge t1 t2) (merge t2 t1)

end
```

4.2.2 Grow-only Counter Module

The generic `CRDT` interface is limited to merge and comparison operations and therefore does not include update operations such as `increment`. To work within this restriction, the `GCounter`, presented in Listing 4.4, adopts a structure using two modules, with the update logic defined separately in an auxiliary module, `GCAuxiliary` in Listing 4.3.

In this auxiliary module, the state representation differs from the vector-based model used in VeriFx. Instead of a vector of entries, the implementation represents the state as a record containing a single integer payload, with convergence defined through the application of the `max` function between states. The `GCounter` module then provides the implementation of the `CRDT` interface, by importing `GCAuxiliary`, it maps the concrete types and functions to the abstract definitions. This separation keeps the `increment` operation accessible in the auxiliary module, while ensuring that `GCounter` satisfies the axioms and signature defined in the interface.

Listing 4.3: The implementation of Auxiliary module in Why3

```
module GCAuxiliary

  use int.Int, int.MinMax

  type payload = int
  type t = { payload: int; }
```

```
7
8   let function increment (v:int) (t:t) : t =
9     { payload = t.payload + v }
10
11  let function merge (t1 t2: t) : t =
12    { payload = max t1.payload t2.payload }
13
14  end
```

Listing 4.4: The implementation of Grow-only Counter in Why3

```
1  module GCounter : Crdt
2
3    use int.Int, int.MinMax
4    use GCAuxiliary as GCAux
5
6    type payload = GCAux.payload
7    type t = GCAux.t
8
9    let function get_payload (t: t) : int
10   = GCAux.get_payload t
11
12   let function create () : t
13   = GCAux.create ()
14
15   function merge (t1 t2: t) : t
16   = GCAux.merge t1 t2
17
18   predicate compare (t1 t2: t)
19   = GCAux.compare t1 t2
20
21   predicate equals (t1 t2: t)
22   = compare t1 t2 /\ compare t2 t1
23
24 end
```

4.2.3 Positive-Negative Counter Module

The `PNCounter` module illustrates state composition by combining two internal counters, `incs` and `decs`. A distinguishing aspect of this translation is the enforcement of internal consistency through invariants, in contrast with the dynamic computation used in VeriFx.

In the VeriFx implementation, the observable value is computed dynamically. In Why3, the state definition shown in Listing 4.5 includes a `payload_pn` field constrained by a strong invariant, `payload_pn = incs - decs`. This invariant requires that, for any valid instance, the payload is mathematically equal to the difference between the two internal counters. The verification system prevents the existence of any state that violates this property.

To preserve this consistency, the implementation adopts a correct-by-construction approach using the helper function `make`. This function receives the two auxiliary states, computes the corresponding payload, and creates the new object. Operations such as `increment`, `decrement`, and `merge` therefore rely on `make` for object creation, ensuring that every state transition preserves the structural invariant.

In summary, the formalization of the PN-Counter validates the feasibility of translating CRDT logic from an automated environment to a deductive one. While VeriFx efficiently handles algebraic convergence properties using SMT solvers, a deductive approach is essential for capturing the internal semantics that automated tools often omit. This comparison highlights that for complex structures, the rigor of Why3 complements the efficiency of VeriFx, providing a depth of verification that ensures not just replica convergence, but also the logical correctness of the data they hold.

Listing 4.5: The implementation of Positive-Negative Counter in Why3

```
module PNCounter : Crdt

  use int.Int
  use GCAuxiliary as GCAux

  type payload = int
  type t = { incs: GCAux.t; decs: GCAux.t; payload_pn: payload }
    invariant { payload_pn = GCAux.get_payload incs - GCAux.get_payload decs }
    by { payload_pn = 0; incs = GCAux.create (); decs = GCAux.create (); }
  . . .
  let function make (t1 t2: GCAux.t) : t
  = let v = GCAux.get_payload t1 - GCAux.get_payload t2 in
    { incs = t1; decs = t2; payload_pn = v }

  function increment (v: int) (t: t) : t
  = make (GCAux.increment v t.incs) t.decs

  function decrement (v: int) (t: t) : t
  = make t.incs (GCAux.increment v t.decs)
  . . .
  let function merge (t1 t2: t) : t
  = make (GCAux.merge t1.incs t2.incs) (GCAux.merge t1.decs t2.decs)

end
```

5

Work Plan

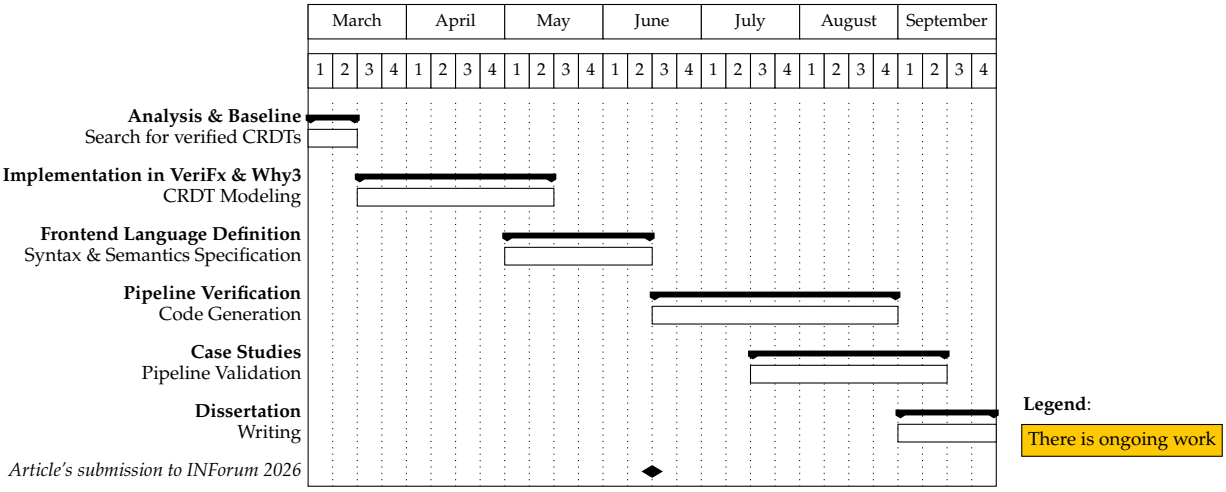

Figure 5.1: Planned Schedule

- **Task 1: Analysis & Baseline** In the first two weeks of March, our goal is to identify and analyze existing formally verified CRDTs across different languages. The goal is to select a set of CRDTs that will serve as our baseline subjects of interest, regardless of the tools originally used to verify them.

- **Task 2: Implementation in VeriFx & Why3** Afterwards, over the next five weeks, we will implement and verify the set of CRDTs identified in the previous task using both VeriFx and Why3. This will allow us to establish a ground truth and understand the specific challenges of verifying these structures.

- **Task 3: Frontend Language Definition** Over two months, starting in the middle of April, we plan on defining a frontend language for describing CRDTs. In this task, we will specify both the syntax and semantics of the language, ensuring it can capture the necessary logic.

- **Task 4: Pipeline Verification** The main objective is to design and implement translation rules that convert the code written in our frontend language into valid inputs for VeriFx and Why3.
- **Task 5: Case Studies** Running in parallel with the pipeline development, this task aims to validate our solution. We will select complex scenarios and pass them through the developed pipeline to demonstrate that our language can successfully describe CRDTs.
- **Task 6: Dissertation** We will dedicate the entire month of September to writing the dissertation, consolidating the results, and refining the documentation of the developed tool.

Grow-only Counter

```
1  import org.verifx.practical.crdts.CvRDT
2  import org.verifx.practical.crdts.CvRDTProof
3
4  class GCounter(entries: Vector[Int]) extends CvRDT[GCounter] {
5      pre increment(replica: Int) {
6              replica >= 0 \&\&
7              replica < this.entries.size
8          }
9
10     def increment(replica: Int) = {
11         val count = this.entries.get(replica)
12         new GCounter(this.entries.write(replica, count + 1))
13     }
14
15     @recursive
16     private def computeValue(sum: Int = 0, replica: Int = 0): Int = {
17         if (replica >= 0 \&\& replica < this.entries.size) {
18                 val count = this.entries.get(replica)
19                 this.computeValue(sum + count, replica + 1)
20             }
21         else
22             sum
23     }
24
25     def value() = this.computeValue()
26
27     private def max(t: Tuple[Int, Int]) = if (t.fst >= t.snd) t.fst else t.snd
28
29     def merge(that: GCounter): GCounter = {
30         val mergedEntries = this.entries.zip(that.entries).map(this.max \_)
31         new GCounter(mergedEntries)
32     }
33
34     def compare(that: GCounter): Boolean = {
35         this.entries
36             .zip(that.entries)
```

```
37            .forall((tup: Tuple[Int, Int]) => tup.fst <= tup.snd)
38    }
39 }
40
41 object GCounter extends CvRDTProof[GCounter]
```

Positive-Negative Counter

```
1  class PNCounter(p: GCounter, n: GCounter) extends CvRDT[PNCounter] {
2    def value() = this.p.value() - this.n.value()
3
4    def increment(replica: Int) =
5      new PNCounter(this.p.increment(replica), this.n)
6
7    def decrement(replica: Int) =
8      new PNCounter(this.p, this.n.increment(replica))
9
10   def merge(that: PNCounter) =
11     new PNCounter(this.p.merge(that.p), this.n.merge(that.n))
12
13   def compare(that: PNCounter): Boolean =
14     this.p.compare(that.p) && this.n.compare(that.n)
15 }
16
17 object PNCounter extends CvRDTProof[PNCounter]
```

Grow-only Set

```
1  class GSet[V](set: Set[V] = new Set[V]()) extends CvRDT[GSet[V]] {
2    def add(element: V) = new GSet(this.set.add(element))
3    def lookup(element: V) = this.set.contains(element)
4    def merge(that: GSet[V]) = new GSet(this.set.union(that.set))
5    def compare(that: GSet[V]) = this.set.subsetOf(that.set)
6
7    def exists(f: V => Boolean) = this.set.exists(f)
8  }
9
10 object GSet extends CvRDTProof1[GSet]
```

Two-Phase Set

```scala
import org.verifx.practical.crdts.CvRDT
import org.verifx.practical.crdts.CvRDTProof1

class TwoPSet[V](added: Set[V] = new Set[V](), removed: Set[V] = new Set[V]()) extends
    ↪ CvRDT[TwoPSet[V]] {
  def lookup(element: V) = this.added.contains(element) && !this.removed.contains(element)
  def add(element: V) = new TwoPSet(this.added.add(element), this.removed)
  def remove(element: V) = new TwoPSet(this.added, this.removed.add(element))

  def merge(that: TwoPSet[V]) = {
    val newAdded = this.added.union(that.added)
    val newRemoved = this.removed.union(that.removed)
    new TwoPSet(newAdded, newRemoved)
  }

  def compare(that: TwoPSet[V]) = {
    this.added.subsetOf(that.added) &&
    this.removed.subsetOf(that.removed)
  }
}

object TwoPSet extends CvRDTProof1[TwoPSet]
```

Operation-Based Set

```
1  import org.verifx.practical.crdts.CmRDT
2  import org.verifx.practical.crdts.CmRDTProof2
3
4  class Tag[ID](replica: ID, counter: Int)
5
6  object Op {
7    enum SetOp[V, ID] {
8      Add(e: V) | Remove(e: V)
9    }
10
11   enum SetMsg[V, ID] {
12     AddMsg(e: V, tag: Tag[ID]) | RemoveMsg(e: V, tags: Set[Tag[ID]])
13   }
14 }
15
16 class ORSet[V, ID](myID: ID, counter: Int = 0,
17                    elements: Map[V, Set[Tag[ID]]] = new Map[V, Set[Tag[ID]]]) extends CmRDT[
                       ↪ SetOp[V, ID], SetMsg[V, ID], ORSet[V, ID]] {
18
19   def lookup(e: V) =
20     this.elements.getOrElse(e, new Set[Tag[ID]]).nonEmpty()
21
22   def add(e: V): SetMsg[V, ID] =
23     new AddMsg(e, new Tag(this.myID, this.counter + 1))
24
25   def addDownstream(e: V, tag: Tag[ID]) = {
26     val newCounter = if (tag.replica == this.myID) tag.counter else this.counter
27
28     val tags = this.elements.getOrElse(e, new Set[Tag[ID]])
29     val newTags = tags.add(tag)
30
31     new ORSet(
32       this.myID,
33       newCounter,
34       this.elements.add(e, newTags)
35     )
```

```
36  }
37
38  def preRemove(e: V) = this.lookup(e)
39
40  def remove(e: V): SetMsg[V, ID] =
41    new RemoveMsg[V, ID](e, this.elements.getOrElse(e, new Set[Tag[ID]]))
42
43  def removeDownstream(e: V, tags: Set[Tag[ID]]) = {
44    val observedTags = this.elements.getOrElse(e, new Set[Tag[ID]])
45    val newTags = observedTags.diff(tags)
46    val newElements = this.elements.add(e, newTags)
47
48    new ORSet(
49      this.myID,
50      this.counter,
51      newElements
52    )
53  }
54
55  override def enabledSrc(op: SetOp[V, ID]) = op match {
56    case Add(e) => true
57    case Remove(e) => this.preRemove(e)
58  }
59
60  override def compatibleS(that: ORSet[V, ID]) = {
61    this.myID != that.myID
62  }
63
64  override def compatible(x: SetMsg[V, ID], y: SetMsg[V, ID]) = x match {
65    case AddMsg(_, tag) => {
66      y match {
67        case AddMsg(_, _) => true
68        case RemoveMsg(_, tags) => !tags.contains(tag)
69      }
70    }
71    case RemoveMsg(_, tags) => {
72      y match {
73        case AddMsg(_, tag) => !tags.contains(tag)
74        case RemoveMsg(_, _) => true
75      }
76    }
77  }
78
79  def prepare(op: SetOp[V, ID]) = op match {
80    case Add(e) => this.add(e)
81    case Remove(e) => this.remove(e)
82  }
83
84  def effect(msg: SetMsg[V, ID]) = msg match {
85    case AddMsg(e, tag) => this.addDownstream(e, tag)
```

```
86      case RemoveMsg(e, tags) => this.removeDownstream(e, tags)
87    }
88
89    override def equals(that: ORSet[V, ID]) = {
90      this.elements == that.elements
91    }
92  }
93
94  object ORSet extends CmRDTProof2[SetOp, SetMsg, ORSet]
```

Why3 Implementation

```
module Crdt

  type payload

  type t

  val function create () : t

  val function get_payload (t: t) : payload

  function merge t t : t

  predicate compare t t

  predicate equals (t1 t2: t)
  = compare t1 t2 /\ compare t2 t1

  axiom merge_commutative: forall t1 t2: t.
    equals (merge t1 t2) (merge t2 t1)

end

module GCAuxiliary

  use int.Int, int.MinMax

  type payload = int

  type t = { payload: int; }

  let function get_payload (t: t) : int
  = t.payload

  let function create () : t
  = { payload = 0 }

```

```
37    let function increment (v:int) (t:t) : t =
38      { payload = t.payload + v }
39
40    let function merge (t1 t2: t) : t =
41      { payload = max t1.payload t2.payload }
42
43    predicate compare (t1 t2: t)
44    = t1.payload = t2.payload
45
46  end
47
48  module GCounter : Crdt
49
50    use int.Int, int.MinMax
51    use GCAuxiliary as GCAux
52
53    type payload = GCAux.payload
54
55    type t = GCAux.t
56
57    let function get_payload (t: t) : int
58    = GCAux.get_payload t
59
60    let function create () : t
61    = GCAux.create ()
62
63    function merge (t1 t2: t) : t
64    = GCAux.merge t1 t2
65
66    predicate compare (t1 t2: t)
67    = GCAux.compare t1 t2
68
69    predicate equals (t1 t2: t)
70    = compare t1 t2 /\ compare t2 t1
71
72  end
73
74  module PNCounter : Crdt
75
76    use int.Int
77    use GCAuxiliary as GCAux
78
79    type payload = int
80
81    type t = { incs: GCAux.t; decs: GCAux.t; payload_pn: payload }
82      invariant { payload_pn = GCAux.get_payload incs - GCAux.get_payload decs }
83      by { payload_pn = 0; incs = GCAux.create (); decs = GCAux.create (); }
84
85    let function get_payload (t: t) : payload
86    = t.payload_pn
```

```
87
88    let function create () : t
89    = { incs = GCAux.create (); decs = GCAux.create (); payload_pn = 0 }
90
91    let function make (t1 t2: GCAux.t) : t
92    = let v = GCAux.get_payload t1 - GCAux.get_payload t2 in
93      { incs = t1; decs = t2; payload_pn = v }
94
95    function increment (v: int) (t: t) : t
96    = make (GCAux.increment v t.incs) t.decs
97
98    function decrement (v: int) (t: t) : t
99    = make t.incs (GCAux.increment v t.decs)
100
101   predicate compare (t1 t2: t)
102   = GCAux.compare t1.incs t2.incs /\
103     GCAux.compare t1.decs t2.decs
104
105   predicate equals (t1 t2: t)
106   = compare t1 t2 /\ compare t2 t1
107
108   let function merge (t1 t2: t) : t
109   = make (GCAux.merge t1.incs t2.incs) (GCAux.merge t1.decs t2.decs)
110
111 end
```

Bibliography

[1] C. W. Barrett et al. "CVC4". In: *Computer Aided Verification - 23rd International Conference, CAV 2011, Snowbird, UT, USA, July 14-20, 2011. Proceedings*. Ed. by G. Gopalakrishnan and S. Qadeer. Vol. 6806. Lecture Notes in Computer Science. Springer, 2011, pp. 171–177. DOI: 10.1007/978-3-642-22110-1_14 (cit. on pp. 14, 19).

[2] J. Bauwens and E. G. Boix. "Nested Pure Operation-Based CRDTs". In: *37th European Conference on Object-Oriented Programming, ECOOP 2023, Seattle, Washington, United States, July 17-21, 2023*. Ed. by K. Ali and G. Salvaneschi. Vol. 263. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023, 2:1–2:26. DOI: 10.4230/LIPICS.ECOOP.2023.2 (cit. on pp. 9, 11, 16).

[3] Y. Bertot and P. Castéran. *Interactive Theorem Proving and Program Development - Coq'Art: The Calculus of Inductive Constructions*. Texts in Theoretical Computer Science. An EATCS Series. Springer, 2004. ISBN: 978-3-642-05880-6. DOI: 10.1007/978-3-662-07964-5 (cit. on p. 19).

[4] D. Borrego et al. "Ensuring Convergence and Invariants Without Coordination". In: *39th European Conference on Object-Oriented Programming, ECOOP 2025, Bergen, Norway, June 30 - July 2, 2025*. Ed. by J. Aldrich and A. Silva. Vol. 333. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2025, 4:1–4:29. DOI: 10.4230/LIPICS.ECOOP.2025.4 (cit. on pp. 5, 12).

[5] S. Conchon et al. "Alt-Ergo 2.2". In: *SMT Workshop: International Workshop on Satisfiability Modulo Theories*. 2018 (cit. on pp. 14, 19).

[6] G. DeCandia et al. "Dynamo: amazon's highly available key-value store". In: *Proceedings of the 21st ACM Symposium on Operating Systems Principles 2007, SOSP 2007, Stevenson, Washington, USA, October 14-17, 2007*. Ed. by T. C. Bressoud and M. F. Kaashoek. ACM, 2007, pp. 205–220. DOI: 10.1145/1294261.1294281 (cit. on p. 8).

[7] J. Filliâtre and A. Paskevich. "Why3 - Where Programs Meet Provers". In: *Programming Languages and Systems - 22nd European Symposium on Programming, ESOP 2013, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2013, Rome, Italy, March 16-24, 2013. Proceedings*. Ed. by M. Felleisen and P. Gardner. Vol. 7792. Lecture Notes in Computer Science. Springer, 2013, pp. 125–128. DOI: 10.1007/978-3-642-37036-6_8 (cit. on pp. 1, 2, 13, 14, 19, 24).

[8] S. Gilbert and N. A. Lynch. "Perspectives on the CAP Theorem". In: *Computer* 45.2 (2012), pp. 30–36. DOI: 10.1109/MC.2011.389 (cit. on pp. 1, 4).

[9] M. Kleppmann and A. R. Beresford. "A Conflict-Free Replicated JSON Datatype". In: *IEEE Trans. Parallel Distributed Syst.* 28.10 (2017), pp. 2733–2746. DOI: 10.1109/TPDS.2017.2697382 (cit. on p. 11).

[10] M. Köhler et al. "Consistent Local-First Software: Enforcing Safety and Invariants for Local-First Applications". In: *IEEE Trans. Software Eng.* 51.1 (2025), pp. 53–65. DOI: 10.1109/TSE.2024.3477723 (cit. on p. 8).

[11] L. Lamport. "Time, clocks, and the ordering of events in a distributed system". In: *Concurrency: the Works of Leslie Lamport*. Ed. by D. Malkhi. ACM, 2019, pp. 179–196. DOI: 10.1145/3335772.3335934 (cit. on p. 17).

[12] L. M. de Moura and N. S. Bjørner. "Z3: An Efficient SMT Solver". In: *Tools and Algorithms for the Construction and Analysis of Systems, 14th International Conference, TACAS 2008, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2008, Budapest, Hungary, March 29-April 6, 2008. Proceedings*. Ed. by C. R. Ramakrishnan and J. Rehof. Vol. 4963. Lecture Notes in Computer Science. Springer, 2008, pp. 337–340. DOI: 10.1007/978-3-540-78800-3_24 (cit. on pp. 13, 14, 17, 19).

[13] T. Nipkow, L. C. Paulson, and M. Wenzel. *Isabelle/HOL - A Proof Assistant for Higher-Order Logic*. Vol. 2283. Lecture Notes in Computer Science. Springer, 2002. ISBN: 3-540-43376-7. DOI: 10.1007/3-540-45949-9 (cit. on p. 13).

[14] K. D. Porre, C. Ferreira, and E. G. Boix. "VeriFx: Correct Replicated Data Types for the Masses". In: *37th European Conference on Object-Oriented Programming, ECOOP 2023, Seattle, Washington, United States, July 17-21, 2023*. Ed. by K. Ali and G. Salvaneschi. Vol. 263. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023, 9:1–9:45. DOI: 10.4230/LIPICS.ECOOP.2023.9 (cit. on pp. 1, 12, 13, 17, 21).

[15] N. M. Preguiça. "Conflict-free Replicated Data Types: An Overview". In: *CoRR* abs/1806.10254 (2018). arXiv: 1806.10254 (cit. on pp. 5, 7).

[16] A. d. R. M. Rijo. "Building Tunable CRDTs". MA thesis. Universidade NOVA de Lisboa (Portugal), 2018 (cit. on p. 9).

[17] D. dos Santos Borrego. "Verifying and Enforcing Application Constraints in Antidote SQL". MA thesis. Universidade NOVA de Lisboa (Portugal), 2022 (cit. on p. 2).

[18] M. Shapiro et al. "Convergent and Commutative Replicated Data Types". In: *Bull. EATCS* 104 (2011), pp. 67–88. URL: http://eatcs.org/beatcs/index.php/beatcs/article/view/120 (cit. on pp. 1, 6–9, 15, 16, 21–23).

[19] *The Why3 platform, version 1.8*. URL: https://www.why3.org/doc/ (visited on 2026-02-04) (cit. on pp. 2, 13–15, 24).

[20] E. Yanakieva et al. "Access Control Conflict Resolution in Distributed File Systems using CRDTs". In: *PaPoC@EuroSys 2021, 8th Workshop on Principles and Practice of Consistency for Distributed Data, Online Event, United Kingdom, April 26, 2021*. ACM, 2021, 1:1–1:3. DOI: 10.1145/3447865.3457970 (cit. on p. 13).